



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

CLOJURE PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Programming Languages

TOTAL: 10 QUESTIONS

Q1 What are the primary design goals of Clojure?

Threat Model

Q6 How does Clojure implement object-oriented or functional programming?

Functionalability

Q2 How does Clojure handle type checking: static or dynamic?

Dynamic Type

Q7 How does Clojure handle errors?

Lifecycle

Q3 How does Clojure manage memory?

Configuration

Q8 What testing tools are commonly used in Clojure?

Compliance

Q4 What is Clojure's concurrency model?

Hardening

Q9 What makes Clojure well-suited for its target domain?

Testing

Q5 How are packages and dependencies managed in Clojure?

ESD / IDP

Q10 What are common performance pitfalls in Clojure?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Clojure was designed with specific priorities around

Mitigation

Clojure was designed with specific priorities around performance, safety, or developer productivity depending on its domain. These goals shape its syntax, type system, and standard library choices.

A6 Clojure supports classes and inheritance for OOP,

Observability

Clojure supports classes and inheritance for OOP, or first-class functions and immutability for FP, or both. Many modern Clojure programs blend both paradigms depending on the problem.

A2 Clojure uses its type system to catch

Primitives

Clojure uses its type system to catch errors at compile time (static) or at runtime (dynamic). This affects refactoring safety, tooling support, and the kinds of bugs that reach production.

A7 Clojure surfaces errors via exceptions, Result/Either types,

Configuration

Clojure surfaces errors via exceptions, Result/Either types, or error return values. The choice impacts whether errors are checked at compile time and how calling code is structured.

A3 Clojure uses one of three approaches: manual

Hardening

Clojure uses one of three approaches: manual allocation/deallocation, a garbage collector that periodically reclaims unreachable objects, or automatic reference counting. Each has different latency and throughput tradeoffs.

A8 Clojure has a standard or widely adopted

Governance

Clojure has a standard or widely adopted test runner and assertion library. Tests are typically organized into unit, integration, and end-to-end layers with coverage reporting built in or via plugins.

A4 Clojure supports concurrency through threads, coroutines,

Timephases

Clojure supports concurrency through threads, coroutines, async/await, or an actor model. The choice determines how I/O-bound and CPU-bound workloads are scaled and how shared state is safely accessed.

A9 Clojure excels in its domain due to

Assessment

Clojure excels in its domain due to a combination of performance characteristics, ecosystem maturity, language ergonomics, and community support that makes common tasks simple.

A5 Clojure uses a package manager and a

Federation

Clojure uses a package manager and a manifest file to declare dependencies and version constraints. A lock file pins exact versions to ensure reproducible builds across environments.

A10 Typical performance issues in Clojure include excessive

Optimization

Typical performance issues in Clojure include excessive allocations, blocking the main thread or event loop, O(n^2) data structures, and missing caching. Profiling and benchmarking tools are available to diagnose them.